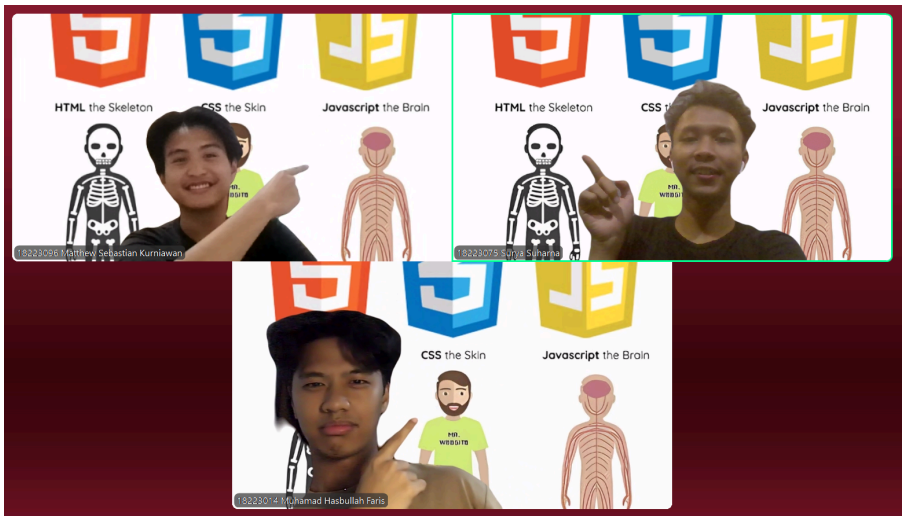
Laporan Tugas Besar 2

IF2211 - Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada Pohon Document Object Model



Disusun oleh:

Kelompok : Karna

Muhamad Hasbullah Faris	18223014
Surya Suharna	18223075
Matthew Sebastian Kurniawan	18223096

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

Daftar Isi 2
I. Deskripsi Tugas 3
II. Landasan Teori 4
2.1 Penjelajahan Graf 4
2.2 Algoritma Breadth-First Search (BFS) 4
2.3 Algoritma Depth-First Search (DFS) 5
III. Aplikasi BFS dan DFS 7
3.1 Pemetaan Pohon DOM (Document Object Model) 7
3.2 Aplikasi Algoritma Breadth-First Search (BFS) 7
3.3 Aplikasi Algoritma Depth-First Search (DFS) 8
3.4 Manajemen Konkurensi dan Optimasi 8
3.5 Antarmuka Pengguna dan Visualisasi Animasi 9
IV. Implementasi dan pengujian 10
4.1 Implementasi Program 10
4.1.1 Package handler 10
4.1.2 Package models 17
4.1.3 Package parser 20
4.1.4 Package traversal 25
4.2 Pengujian 45
4.2.1 BFS 45
4.2.2 DFS 46
4.2.3 LCA dengan Binary Lifting 47
4.3 Analisis Efisiensi dan Perbandingan 48
V. Kesimpulan dan Saran 50
5.1 Kesimpulan 50
5.2 Saran 51
VI. Lampiran 52
VII. Pernyataan Kejujuran 54
VIII. Daftar Pustaka 55

I. Deskripsi Tugas

Document Object Model (DOM) adalah representasi hierarkis dari dokumen HTML dalam bentuk tree (pohon) yang mendefinisikan hubungan antar elemen seperti parent, child, dan sibling. Representasi terstruktur ini memungkinkan setiap elemen pada halaman web diakses dan dimanipulasi secara terprogram, termasuk untuk keperluan visualisasi antarmuka yang dideklarasikan melalui mekanisme target pencarian bernama CSS Selector.

Pada Tugas Besar 2 IF2211 Strategi Algoritma ini, kami ditugaskan untuk mengimplementasikan algoritma pencarian graf, yaitu Breadth First Search (BFS) dan Depth First Search (DFS), dalam menelusuri struktur pohon DOM. Penelusuran mandiri ini bertujuan untuk menjelajahi dan menemukan elemen-elemen HTML spesifik yang cocok dengan target CSS Selector yang diinputkan secara tepat dan terukur.

Aplikasi yang dibangun merupakan aplikasi berbasis web dengan arsitektur Client-Server. Secara spesifik, tugas ini menuntut pemenuhan spesifikasi berikut:

1. **Input:** Aplikasi mampu melakukan scraping HTML dari target URL website yang diberikan, atau menerima raw text HTML secara langsung. Selain itu, aplikasi menerima input berupa pilihan algoritma traversal (BFS atau DFS), target CSS Selector, dan batasan jumlah hasil (Top N atau semua kemunculan).
2. **Pemrosesan (Backend):** Program melakukan parsing dokumen HTML mentah menjadi struktur data pohon (DOM Tree), lalu menjalankan algoritma pencarian yang dipilih beserta pencocokan (matching) terhadap aturan CSS Selector yang diinputkan.
3. **Output (Frontend):** Aplikasi memvisualisasikan struktur pohon DOM secara interaktif yang disertai dengan informasi kedalaman maksimum. Aplikasi juga memberikan sorotan visual (*highlight*) pada jalur yang ditelusuri algoritma untuk menandakan elemen yang terpengaruh, menampilkan traversal log yang merekam tahapan penelusuran, serta menyajikan metrik analitik berupa waktu pencarian dan total node yang dikunjungi.

II. Landasan Teori

2.1 Penjelajahan Graf

Algoritma traversal graf adalah algoritma yang mengunjungi simpul-simpul di dalam graf dengan cara yang sistematik untuk melakukan pencarian solusi persoalan yang direpresentasikan dengan graf. Algoritma pencarian solusi berbasis graf terbagi menjadi dua kelompok besar, yaitu pencarian tanpa informasi (uninformed/blind search) dan pencarian dengan informasi (informed search).

Pencarian tanpa informasi adalah pencarian di mana tidak ada informasi tambahan yang disediakan selain definisi masalah itu sendiri, contohnya seperti Breadth-First Search (BFS), Depth-First Search (DFS), Depth-Limited Search, Iterative Deepening Search, dan Uniform Cost Search. Sebaliknya, pencarian dengan informasi menggunakan heuristik untuk mengetahui non-goal state mana yang lebih menjanjikan daripada yang lain, contohnya seperti algoritma Best First Search dan A*.

2.2 Algoritma Breadth-First Search (BFS)

Algoritma Breadth-First Search (BFS) adalah algoritma pencarian yang melebar. Pencarian dimulai dari sebuah simpul dan menelusuri semua simpul yang berdekatan secara komprehensif, dengan kata lain, algoritma ini menelusuri semua simpul pada kedalaman n terlebih dahulu sebelum beralih menelusuri simpul pada kedalaman n+1.

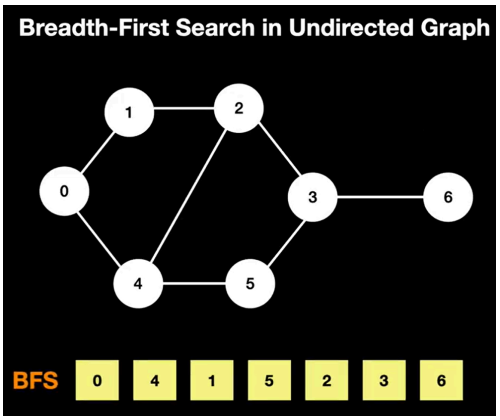
Mengingat struktur data graph (berbeda dengan tree) memungkinkan adanya siklus yang membuat simpul yang sama dikunjungi kembali, algoritma ini memanfaatkan struktur data queue dan look-up table untuk melacak simpul anak yang ditemukan tetapi belum ditelusuri.

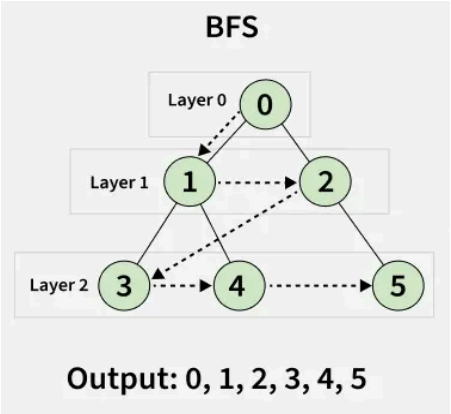
Secara algoritmik, tahapan BFS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul sumber v yang diberikan ke dalam queue dan tandai sebagai telah dikunjungi.

2. **Eksplorasi:** Selama queue tidak kosong, keluarkan suatu simpul dari queue dan kunjungi simpul tersebut. Kemudian, masukkan tetangga ke dalam queue untuk setiap tetangga yang belum dikunjungi dari simpul yang dikeluarkan, dan tandai simpul tersebut sebagai telah dikunjungi.

3. **Basis:** Ulangi langkah eksplorasi hingga queue kosong atau ditemukan simpul target.





BFS umumnya dapat digunakan untuk mencari jarak terpendek antara dua node dengan asumsi jarak antar node sama atau graf tidak berbobot. Di balik kelebihannya yang menjamin solusi optimal atau terpendek, BFS memiliki kekurangan yakni membutuhkan memori yang cukup banyak karena harus menyimpan semua simpul yang dieksplorasi.

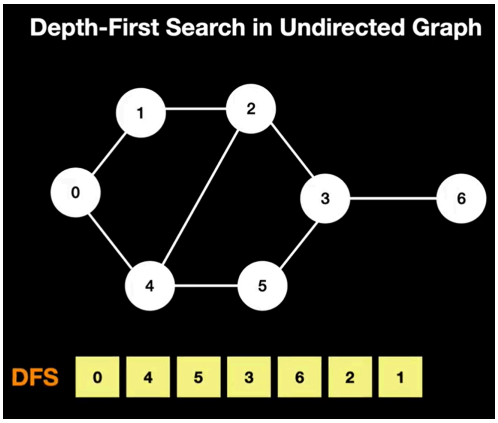
2.3 Algoritma Depth-First Search (DFS)

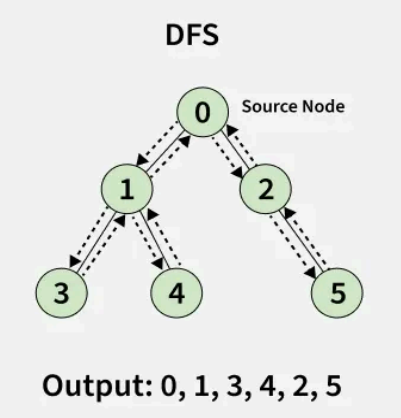
Algoritma Depth-First Search (DFS) adalah algoritma pencarian yang mendalam. Algoritma ini dimulai dari simpul awal, lalu dilanjutkan dengan hanya mengunjungi simpul anak paling kiri pada tingkat selanjutnya, dan baru dilanjutkan pada node sebelah kanan.

Serupa dengan BFS, DFS juga menghadapi kendala siklus pada struktur data graf, sehingga DFS memanfaatkan struktur data stack dan look-up table dalam prosesnya untuk melacak simpul anak yang ditemukan tetapi belum ditelusuri. Jika solusi ditemukan pada suatu jalur, DFS tidak memerlukan proses backtracking (penelusuran balik) untuk mendapatkan jalur yang diinginkan.

Secara algoritmik, tahapan DFS dapat diurai menjadi:

1. **Inisialisasi:** Masukkan simpul sumber v yang diberikan ke dalam stack dan tandai sebagai telah dikunjungi.
2. **Eksplorasi:** Selama stack tidak kosong, keluarkan suatu simpul dari stack dan kunjungi simpul tersebut. Kunjungi simpul w yang bertetangga dengan simpul v dan masukkan ke stack, lalu ulangi sampai mencapai simpul u yang tidak memiliki tetangga yang belum dikunjungi.
3. **Backtrack:** Ketika mencapai simpul u (titik buntu), lakukan pencarian runut-balik (backtrack) ke simpul terakhir yang dikunjungi sebelumnya dan mempunyai simpul w yang belum dikunjungi.
4. **Basis:** Pencarian berakhir bila tidak ada lagi simpul yang belum dikunjungi yang dapat dicapai dari simpul yang telah dikunjungi atau ditemukan simpul target.





Keunggulan DFS terletak pada penggunaan memori yang relatif lebih kecil dibandingkan BFS. Namun, kekurangannya adalah tidak ada jaminan untuk menemukan solusi yang terbaik atau optimal. Dalam Directed Acyclic Graphs (DAG), DFS juga dapat digunakan untuk melakukan pengurutan topologi yang sangat penting dalam penjadwalan dan manajemen prasyarat (prerequisites).

III. Aplikasi BFS dan DFS

Penerapan algoritma *Breadth-First Search* (BFS) dan *Depth-First Search* (DFS) dalam mesin pencari ini berfokus pada penelusuran dan ekstraksi data dari struktur dokumen HTML. Untuk mengaplikasikan kedua algoritma ini, diperlukan tahapan pemetaan dokumen menjadi struktur pohon yang valid, dilanjutkan dengan strategi logika penelusuran, serta penerapan manajemen konkurensi. Bab ini akan menguraikan ketiga tahapan tersebut.

3.1 Pemetaan Pohon DOM (Document Object Model)

Sebelum algoritma pencarian dapat dieksekusi, dokumen HTML mentah harus dipetakan terlebih dahulu ke dalam struktur pohon DOM. Dalam representasi komputasi ini, setiap elemen HTML diinstansiasi sebagai sebuah simpul. Setiap simpul bertugas menyimpan sekumpulan atribut elemen seperti tag, identitas (ID), dan kelas, serta memuat referensi *pointer* hierarkis menuju simpul induk (*parent*) dan simpul anak (*children*). Ketersediaan referensi relasional inilah yang menjadi fondasi operasional bagi mesin pencari untuk mengevaluasi kueri selektor CSS (termasuk operator *sibling* dan *ancestor*).

3.2 Aplikasi Algoritma Breadth-First Search (BFS)

Pada aplikasinya, algoritma BFS diterapkan untuk menelusuri pohon DOM secara melebar (*level-order*). Mekanisme dasar dari algoritma ini adalah mengevaluasi seluruh elemen pada tingkat kedalaman yang sama sebelum sistem mulai memproses level berikutnya. Strategi penelusuran BFS ini sangat ideal untuk mengeksekusi struktur DOM yang cenderung lebar, karena hasil pertama yang divalidasi dipastikan merupakan elemen target yang berlokasi terdekat dari *root*.

Dalam implementasi teknisnya, penelusuran BFS ini dijalankan secara paralel, di mana pemrosesan setiap simpul pada level aktif didistribusikan ke *goroutine* yang terpisah. Untuk menjaga konsistensi urutan level, transisi

antar-level pada proses komputasi ini diatur menggunakan kombinasi struktur data *queue* dan mekanisme sync.WaitGroup.

3.3 Aplikasi Algoritma Depth-First Search (DFS)

Sebagai alternatif pencarian, aplikasi DFS beroperasi dengan memprioritaskan penelusuran elemen secara mendalam. Algoritma ini menyusuri satu cabang spesifik secara vertikal dari akar hingga mencapai simpul daun (*leaf node*). Setelah mencapai batas kedalaman tersebut, setelah itu sistem melakukan *backtracking* untuk mengevaluasi elemen pada cabang-cabang lain yang belum dikunjungi.

Untuk menyeimbangkan utilisasi sumber daya dan efisiensi memori, komputasi DFS ini diimplementasikan dengan strategi *hybrid*. Saat kapasitas *worker pool* memadai, sistem akan mengeksekusi eksplorasi simpul anak secara paralel dalam *goroutine* baru. Namun, jika ketersediaan memori terbatas, algoritma secara adaptif beralih ke metode komputasi rekursif konvensional di dalam *goroutine* yang sama. Transisi dinamis ini sangat krusial untuk mencegah terjadinya beban *overhead* akibat potensi ledakan jumlah *thread* yang dapat menyebabkan kegagalan sistem.

3.4 Manajemen Konkurensi dan Optimasi

Sebagai penyempurnaan, eksekusi dari kedua algoritma di atas dikendalikan oleh mekanisme konkurensi terstruktur. Untuk mencegah beban komputasi berlebih, sistem membatasi jumlah *thread* maksimal menggunakan mekanisme *semaphore* yang dikonfigurasi sesuai dengan jumlah inti CPU (*CPU cores*) pada mesin. Di samping itu, optimasi penghentian dini (*early termination*) diterapkan secara global melalui *atomic flag*. Sesaat setelah sistem berhasil menemukan kecocokan elemen yang memenuhi kuota parameter ResultCount, seluruh sisa proses penelusuran di cabang pohon mana pun akan diterminasi secara otomatis. Mekanisme ini secara drastis menghemat pemakaian sumber daya memori dan mengakselerasi waktu respons sistem.

3.5 Antarmuka Pengguna dan Visualisasi Animasi

Untuk memvisualisasikan proses komputasi, sistem dilengkapi dengan antarmuka pengguna berbasis React 19 dan TypeScript. Pemodelan visual pohon DOM akan memanfaatkan *library* React Flow sebagai kanvas *rendering* dan ELK.js untuk automasi *layouting*. Melalui antarmuka ini, pengguna dapat mengonfigurasi masukan berupa tautan URL (yang diproses oleh modul *scraper*) atau kode HTML mentah, beserta kueri selektor CSS dan parameter algoritma. Data tersebut ditransmisikan ke *backend* melalui protokol HTTP POST, yang kemudian memberikan respons berupa objek JSON berisi struktur pohon DOM lengkap, daftar simpul target, rekam jejak *traversal*, serta metrik performa.

Fitur utama dari antarmuka ini adalah visualisasi animasi progresif yang menampilkan simpul-simpul pohon secara berurutan dengan opsi kontrol kecepatan. Karakteristik komputasi akan divisualisasikan melalui arah pergerakan animasi, pada BFS, simpul dirender berdasarkan parameter kedalaman sehingga sorotan bergerak melebar secara bertahap per level, sementara pada DFS, urutan animasi sesuai dengan rekam jejak *pre-order* sehingga sorotan bergerak vertikal ke simpul daun terdalam sebelum melakukan *backtracking*. Diperkuat dengan transisi visual koordinat dan penandaan warna khusus pada simpul target, animasi ini dirancang sebagai instrumen validasi yang menerjemahkan abstraksi logika algoritma ke dalam representasi grafis yang interaktif.

IV. Implementasi dan pengujian

4.1 Implementasi Program

Algoritma pencarian Breadth-First Search (BFS) dan Depth-First Search (DFS) diimplementasikan dengan bahasa pemrograman Golang dan *framework* Next.js. Dalam program ini, terdapat 4 *package* utama yang menyusun logic program, yakni handler, models, parser, dan traversal.

Selain 4 *package* internal yang kami buat, program ini juga menggunakan satu external library, yaitu golang.org/x/net/html, yang berfungsi untuk membantu proses parsing HTML mentah menjadi struktur DOM tree sebelum dilakukan traversal. Digunakan juga *library* reactflow yang digunakan untuk memvisualisasikan *nodes* secara *live* pada *frontend*.

4.1.1 *Package* handler

1. *File* lca_handler.go

```
package handler

import (
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
    "time"

    "tubes2-backend/internal/models"
    "tubes2-backend/internal/parser"
    "tubes2-backend/internal/traversal"
)

func HandleLCA(w http.ResponseWriter, r *http.Request) {
    // Setup CORS
    w.Header().Set("Access-Control-Allow-Origin", "*")
    w.Header().Set("Access-Control-Allow-Methods", "POST, OPTIONS")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type")
```

```
    if r.Method == http.MethodOptions {
        w.WriteHeader(http.StatusOK)
        return
    }
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }

    var req models.LCARequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Format JSON tidak valid: "+err.Error(),
http.StatusBadRequest)
        return
    }

    if strings.TrimSpace(req.SelectorA) == "" ||
strings.TrimSpace(req.SelectorB) == "" {
        http.Error(w, "selector_a dan selector_b wajib diisi",
http.StatusBadRequest)
        return
    }

    start := time.Now()

    var rawHTML string
    var err error
    switch {
    case req.URL != "":
        rawHTML, err = parser.FetchHTML(req.URL)
        if err != nil {
            http.Error(w, fmt.Sprintf("Gagal fetch URL: %v", err),
http.StatusInternalServerError)
            return
        }
    case req.RawHTML != "":
        rawHTML = req.RawHTML
```

```
    default:
        http.Error(w, "URL atau RawHTML tidak boleh kosong",
http.StatusBadRequest)
        return
    }

    domTree, err := parser.ParseHTML(rawHTML)
    if err != nil {
        http.Error(w, fmt.Sprintf("Gagal parsing HTML: %v", err),
http.StatusInternalServerError)
        return
    }

    nodeA, nodeB := traversal.FindFirstTwoMatches(domTree,
req.SelectorA, req.SelectorB)
    if nodeA == nil {
        http.Error(w, fmt.Sprintf("Tidak ada node yang cocok dengan
selector_a: %q", req.SelectorA), http.StatusNotFound)
        return
    }
    if nodeB == nil {
        http.Error(w, fmt.Sprintf("Tidak ada node yang cocok dengan
selector_b: %q", req.SelectorB), http.StatusNotFound)
        return
    }

    lcaNode := traversal.GetLCAByParent(nodeA, nodeB)

    executionTime := float64(time.Since(start).Microseconds()) / 1000.0

    resp := models.LCAResponse{
        ExecutionTimeMs: executionTime,
        NodeA:           nodeA,
        NodeB:           nodeB,
        LCA:             lcaNode,
        FullTree:        domTree,
    }
```

```
    w.Header().Set("Content-Type", "application/json")
    if err := json.NewEncoder(w).Encode(resp); err != nil {
        http.Error(w, "Gagal menyusun response JSON",
http.StatusInternalServerError)
    }
}
```

Fungsi HandleLCA berperan sebagai endpoint HTTP untuk memproses pencarian Lowest Common Ancestor (LCA) pada pohon DOM HTML. Pada awal eksekusi, fungsi ini mengatur header CORS agar request lintas origin dapat diterima, termasuk dukungan untuk metode POST dan preflight OPTIONS. Jika request menggunakan OPTIONS, server langsung mengembalikan status berhasil. Jika metode selain POST digunakan, proses dihentikan dengan respons Method not allowed.

Setelah validasi metode, body request dibaca dan di-decode ke dalam struktur LCARequest. Fungsi kemudian memastikan selector_a dan selector_b terisi; jika salah satu kosong, request ditolak sebagai Bad Request. Selanjutnya sistem menentukan sumber dokumen HTML dari dua opsi, yaitu URL (diambil melalui parser.FetchHTML) atau RawHTML (langsung dari payload). Jika keduanya kosong, request dianggap tidak valid. HTML yang diperoleh lalu diparse menjadi struktur DOM menggunakan parser.ParseHTML. Apabila proses fetch atau parse gagal, handler mengembalikan error internal dengan pesan yang sesuai.

Tahap inti dilakukan melalui traversal.FindFirstTwoMatches, yang mencari node untuk selector A dan selector B dalam satu proses traversal dan berhenti ketika keduanya ditemukan. Jika salah satu node tidak ditemukan, handler mengembalikan Not Found yang spesifik terhadap selector terkait. Setelah kedua node tersedia, LCA dihitung menggunakan traversal.GetLCAByParent dengan pendekatan rantai parent. Waktu eksekusi dihitung dalam milidetik, lalu disusun LCAResponse berisi node_a, node_b, lca, serta full_tree sebagai konteks visualisasi. Terakhir, respons dikirim dalam format JSON. Jika encoding gagal, server mengembalikan pesan error internal.

2. *File* traversal_handler.go

```go
package handler

import (
    "encoding/json"
    "fmt"
    "net/http"
    "time"

    "tubes2-backend/internal/models"
    "tubes2-backend/internal/parser"
    "tubes2-backend/internal/traversal"
)

func HandleTraversal(w http.ResponseWriter, r *http.Request) {
    // Setup CORS
    w.Header().Set("Access-Control-Allow-Origin", "*")
    w.Header().Set("Access-Control-Allow-Methods", "POST, OPTIONS")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type")

    if r.Method == http.MethodOptions {
        w.WriteHeader(http.StatusOK)
        return
    }

    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }

    var req models.TraversalRequest
    err := json.NewDecoder(r.Body).Decode(&req)
    if err != nil {
        http.Error(w, "Format JSON tidak valid: "+err.Error(),
http.StatusBadRequest)
        return
    }
```

```go
    startTime := time.Now()

    var rawHTML string
    if req.URL != "" {
        rawHTML, err = parser.FetchHTML(req.URL)
        if err != nil {
            http.Error(w, fmt.Sprintf("Gagal fetch URL: %v", err),
http.StatusInternalServerError)
            return
        }
    } else if req.RawHTML != "" {
        rawHTML = req.RawHTML
    } else {
        http.Error(w, "URL atau RawHTML tidak boleh kosong",
http.StatusBadRequest)
        return
    }

    domTree, err := parser.ParseHTML(rawHTML)
    if err != nil {
        http.Error(w, fmt.Sprintf("Gagal parsing HTML: %v", err),
http.StatusInternalServerError)
        return
    }

    engine := traversal.NewSearchEngine(domTree, req.Selector,
req.Algorithm, req.ResultCount)
    engine.Execute()

    executionTime := float64(time.Since(startTime).Microseconds()) /
1000.0

    resp := models.TraversalResponse{
        ExecutionTimeMs: executionTime,
        NodesVisited:    engine.NodesVisited,
        NodesFound:      len(engine.Matches),
        MatchedNodes:    engine.Matches,
```

```
        TraversalLog:    engine.Log,
        FullTree:        domTree,
    }

    w.Header().Set("Content-Type", "application/json")
    if err := json.NewEncoder(w).Encode(resp); err != nil {
        http.Error(w, "Gagal menyusun response JSON",
http.StatusInternalServerError)
    }
}
```

Fungsi HandleTraversal berperan sebagai endpoint HTTP untuk menjalankan penelusuran traversal pada struktur DOM berdasarkan selector yang diberikan pengguna. Pada tahap awal, fungsi ini melakukan pengaturan CORS agar endpoint dapat menerima akses lintas origin, termasuk mendukung metode POST dan preflight OPTIONS. Jika request menggunakan metode OPTIONS, server akan langsung mengembalikan status berhasil. Namun, apabila metode yang digunakan bukan POST, proses dihentikan dan sistem mengembalikan respons Method not allowed.

Setelah lolos validasi metode, isi body request dibaca dan di-*decode* ke dalam struktur TraversalRequest. Jika format JSON tidak valid, fungsi akan mengembalikan error Bad Request beserta detail kesalahannya. Selanjutnya, fungsi mencatat waktu mulai eksekusi untuk kebutuhan pengukuran performa. Sumber dokumen HTML kemudian ditentukan dari dua kemungkinan masukan, yaitu URL atau RawHTML. Jika URL tersedia, konten HTML diambil melalui parser.FetchHTML; jika tidak, fungsi menggunakan RawHTML secara langsung. Apabila keduanya kosong, request dianggap tidak valid dan proses dihentikan.

Konten HTML yang berhasil diperoleh di-*parse* menjadi pohon DOM melalui parser.ParseHTML. Jika parsing gagal, fungsi mengembalikan error internal server dengan pesan yang sesuai. Setelah pohon DOM tersedia, handler membentuk mesin pencarian traversal menggunakan traversal.NewSearchEngine dengan parameter selector target, algoritma traversal yang dipilih, serta jumlah hasil yang diinginkan. Eksekusi traversal

kemudian dijalankan melalui engine.Execute(), yang menghasilkan kumpulan *node* yang cocok sekaligus log proses penelusuran.

Pada tahap akhir, fungsi menghitung durasi eksekusi dalam satuan milidetik, lalu menyusun objek TraversalResponse yang memuat metrik dan hasil penting, seperti jumlah *node* yang dikunjungi, jumlah *node* yang ditemukan, daftar *node* yang cocok, log traversal, serta representasi penuh pohon DOM. Response dikirim dalam format JSON dengan header Content-Type yang sesuai. Jika terjadi kegagalan saat *encoding* response, handler akan mengembalikan pesan error internal agar kegagalan tersebut tetap terinformasi dengan jelas.

4.1.2 *Package* models

1. *File* dom.go

```go
package models

type Node struct {
    TagName     string            `json:"tag_name"`
    ID          string            `json:"id,omitempty"`
    Classes     string            `json:"class,omitempty"`
    Attributes  map[string]string `json:"attributes,omitempty"`
    TextContent string            `json:"text_content,omitempty"`
    Children    []*Node           `json:"children,omitempty"`
    Parent      *Node             `json:"-"`
}

// payload input (fitur pencarian elemen)
type TraversalRequest struct {
    URL         string `json:"url"`
    RawHTML     string `json:"raw_html"`
    Selector    string `json:"selector"`
    Algorithm   string `json:"algorithm"`
    ResultCount int    `json:"result_count"`
}
```

```go
// format data Output (metrik pencarian dan hasil traverse)
type TraversalResponse struct {
    ExecutionTimeMs float64  `json:"execution_time_ms"`
    NodesVisited    int      `json:"nodes_visited"`
    NodesFound      int      `json:"nodes_found"`
    MatchedNodes    []*Node  `json:"matched_nodes"`
    TraversalLog    []string `json:"traversal_log"`
    FullTree        *Node    `json:"full_tree"`
}

// payload input (fitur pencarian Lowest Common Ancestor)
type LCARequest struct {
    URL       string `json:"url"`
    RawHTML   string `json:"raw_html"`
    SelectorA string `json:"selector_a"`
    SelectorB string `json:"selector_b"`
}

// format data Output (hasil perhitungan LCA)
type LCAResponse struct {
    ExecutionTimeMs float64 `json:"execution_time_ms"`
    NodeA           *Node   `json:"node_a"`
    NodeB           *Node   `json:"node_b"`
    LCA             *Node   `json:"lca"`
    FullTree        *Node   `json:"full_tree"`
}
```

File dom.go berisi definisi struktur data inti yang digunakan untuk merepresentasikan pohon DOM serta format pertukaran data antara client dan server pada fitur traversal dan LCA. Struktur utama yang menjadi fondasi adalah Node, yang merepresentasikan satu simpul HTML dengan atribut seperti TagName, ID, Classes, Attributes, TextContent, daftar Children, serta referensi Parent. Melalui susunan ini, setiap elemen HTML dapat dipetakan ke bentuk tree yang lengkap, sehingga proses pencarian node, penelusuran struktur, dan analisis relasi antar node dapat dilakukan secara sistematis.

Pada proses serialisasi JSON, field Parent sengaja dikecualikan agar respons tidak membentuk referensi siklik dan tetap efisien saat dikirim ke client.

Selain struktur node, file ini juga mendefinisikan payload input untuk fitur traversal melalui TraversalRequest, yang memuat sumber dari URL atau RawHTML, selector target, algoritma traversal yang dipilih, serta jumlah node yang diminta (ResultCount). Hasil eksekusi traversal kemudian dimasukkan ke dalam TraversalResponse, yang memuat metrik performa seperti waktu eksekusi, jumlah node yang dikunjungi, jumlah node yang ditemukan, daftar node hasil kecocokan, jejak proses traversal, dan keseluruhan pohon DOM (FullTree) untuk keperluan visualisasi maupun evaluasi.

Untuk fitur Lowest Common Ancestor, file ini menyediakan struktur input LCARequest yang berisi sumber dari URL atau RawHTML serta dua selector pembanding, yaitu SelectorA dan SelectorB. Adapun hasil perhitungannya disimpan dalam LCAResponse, yang mencakup waktu eksekusi, node hasil selector A, node hasil selector B, node LCA yang ditemukan, serta pohon DOM lengkap sebagai konteks. Dengan demikian, dom.go berfungsi sebagai kontrak data utama yang menjaga konsistensi format request-response sekaligus menjadi dasar interoperabilitas antarmodul pada sistem backend.

Setelah atribut simpul berhasil dipetakan, fungsi mapNode menelusuri simpul anak (*child nodes*) secara rekursif untuk membangun relasi hierarki dua arah. Setiap anak yang valid dimasukkan ke dalam daftar Children simpul induk, sekaligus menetapkan simpul induk tersebut sebagai Parent bagi sang anak. Penetapan referensi Parent (childNode.Parent = myNode) ini menjadi fondasi struktural krusial bagi implementasi fitur tingkat lanjut. Pemetaan *pointer* induk sejak fase parsing memberi keleluasaan bagi mesin pencarian untuk bergerak naik menyusuri hierarki pohon DOM. Relasi dua arah ini tidak hanya memfasilitasi evaluasi selektor CSS kompleks, seperti elemen sibling dan ancestor, saat mengeksekusi *Multithreading* paralel, tetapi juga menjadi prasyarat mutlak bagi pra-pemrosesan *Lowest Common Ancestor* (LCA) dalam membangun tabel algoritma *Binary Lifting* dan merespons query pada kompleksitas waktu optimal, yakni O(log N).

4.1.3 *Package* parser

1. *File* html_parser.go

```
package parser

import (
    "strings"
    "tubes2-backend/internal/models"

    "golang.org/x/net/html"
)

func ParseHTML(rawHTML string) (*models.Node, error) {
    reader := strings.NewReader(rawHTML)

    // menggunakan html.Parse dari package golang.org/x/net/html
    doc, err := html.Parse(reader)
    if err != nil {
        return nil, err
    }

    rootNode := mapNode(doc)
    return rootNode, nil
}

func mapNode(n *html.Node) *models.Node {
    if n == nil {
        return nil
    }
    myNode := &models.Node{}

    switch n.Type {
    case html.ElementNode:
        myNode.TagName = n.Data
        if len(n.Attr) > 0 {
            myNode.Attributes = make(map[string]string, len(n.Attr))
            for _, attr := range n.Attr {
```

```go
                myNode.Attributes[attr.Key] = attr.Val
                if attr.Key == "id" {
                    myNode.ID = attr.Val
                }
                if attr.Key == "class" {
                    myNode.Classes = attr.Val
                }
            }
        }
    case html.TextNode:
        text := strings.TrimSpace(n.Data)
        if text == "" {
            return nil
        }
        myNode.TagName = "#text"
        myNode.TextContent = text
    case html.DocumentNode:
        myNode.TagName = "#document"
    default:
        return nil
    }

    for c := n.FirstChild; c != nil; c = c.NextSibling {
        childNode := mapNode(c)
        if childNode != nil {
            childNode.Parent = myNode
            myNode.Children = append(myNode.Children, childNode)
        }
    }

    return myNode
}
```

File html_parser.go berisi logika utama untuk memproses string HTML mentah menjadi struktur pohon DOM yang terstruktur. Fungsi ParseHTML bertindak sebagai titik masuk (*entry point*) dalam proses transformasi ini dengan menerima masukan berupa teks HTML dan mengubahnya menjadi

aliran data *reader*. Selanjutnya, fungsi ini memanfaatkan pustaka eksternal golang.org/x/net/html untuk melakukan parsing data tersebut menjadi objek dokumen HTML dasar. Setelah proses analisis sintaksis berhasil diselesaikan, dokumen tersebut langsung didelegasikan ke fungsi mapNode untuk diterjemahkan ke dalam representasi memori internal sistem yang berbasis models.Node, lalu mengembalikan *root node* dari pohon yang telah terbentuk secara utuh.

Fungsi mapNode memiliki peran untuk memetakan dan mengonversi objek simpul bawaan pustaka eksternal menjadi struktur models.Node khusus yang dirancang secara spesifik untuk mesin pencari internal. Dalam operasionalnya, fungsi ini mengevaluasi tipe dari setiap simpul yang diterima secara sekuensial. Apabila simpul berupa elemen HTML biasa (ElementNode), fungsi akan mengekstrak nama tag serta memetakan seluruh atributnya ke dalam struktur data map, di mana atribut id dan class diekstrak dan disimpan secara eksplisit untuk mengefisienkan komputasi pada saat pencocokan selektor CSS. Sebaliknya, jika simpul berupa teks (TextNode), fungsi akan memangkas karakter spasi kosong (*whitespace*) dan menyimpannya sebagai konten teks murni, atau mengabaikannya sama sekali jika hanya berisi spasi kosong agar tidak membebani memori, sedangkan untuk simpul dokumen (DocumentNode) akan diberikan penanda tag indikator berupa #document.

2. *File* scraper.go

```
package parser

import (
    "fmt"
    "io"
    "net/http"
    "time"
)

const defaultUserAgent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```go
AppleWebKit/537.36 " +
    "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"

func FetchHTML(targetURL string) (string, error) {
    client := &http.Client{
        Timeout: 15 * time.Second,
    }

    req, err := http.NewRequest(http.MethodGet, targetURL, nil)
    if err != nil {
        return "", fmt.Errorf("URL tidak valid: %v", err)
    }

    req.Header.Set("User-Agent", defaultUserAgent)
    req.Header.Set("Accept",
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8")
    req.Header.Set("Accept-Language", "en-US,en;q=0.9,id;q=0.8")

    resp, err := client.Do(req)
    if err != nil {
        return "", fmt.Errorf("gagal mengakses URL: %v", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return "", fmt.Errorf("status HTTP tidak OK: %d",
resp.StatusCode)
    }

    bodyBytes, err := io.ReadAll(resp.Body)
    if err != nil {
        return "", fmt.Errorf("gagal membaca isi halaman: %v", err)
    }

    return string(bodyBytes), nil
}
```

File scraper.go memuat logika fungsional untuk mengambil representasi teks HTML mentah dari sebuah halaman web melalui jaringan internet. Fungsi FetchHTML bertindak sebagai agen utama dalam proses ini dengan menerima parameter berupa URL tujuan. Pada tahap awal, fungsi menginisialisasi klien HTTP internal yang telah dikonfigurasi dengan batas *timeout* maksimal selama 15 detik. Pembatasan durasi ini diimplementasikan secara preventif untuk mencegah sistem berstatus menggantung (*hang*) atau memonopoli sumber daya aplikasi apabila server tujuan mengalami latensi jaringan yang tinggi maupun kegagalan respons. Setelah klien HTTP disiapkan, fungsi mulai merakit instans *request* berbasis metode GET terhadap URL sasaran.

Sebelum permintaan tersebut dieksekusi, fungsi secara eksplisit memanipulasi spesifikasi header HTTP agar menyerupai lalu lintas peramban konvensional. *Header* User-Agent disuntikkan dengan profil peramban web standar, dalam hal ini meniru Google Chrome pada lingkungan Windows untuk menyimulasikan identitas klien manusia yang sah. Modifikasi ini merupakan taktik teknis yang esensial untuk meminimalisasi risiko pemblokiran akses oleh mekanisme keamanan anti-bot atau *firewall* yang kerap diterapkan pada server target. Selain itu, penambahan *header* Accept dan Accept-Language juga ditautkan untuk menegaskan spesifikasi negosiasi konten, memastikan server mengembalikan dokumen dalam format HTML yang dapat diproses.

Setelah proses transmisi permintaan selesai, fungsi akan langsung mengevaluasi respons yang dikembalikan oleh server tujuan. Validasi dilakukan terhadap kode status HTTP. Apabila server merespons dengan status selain dari 200 OK, operasi akan langsung dibatalkan dengan melempar pesan *error*. Sebaliknya, pada respons yang valid, aliran data pada bodi balasan akan diekstraksi secara menyeluruh menggunakan utilitas pembaca dari paket io, untuk kemudian dikonversi ke dalam format *string*. Selama tahapan ekstraksi ini, fungsi memanfaatkan instruksi defer untuk menjamin penutupan kanal bodi respons setelah komputasi selesai, sehingga aplikasi tetap aman dari risiko *memory leak*. Hasil akhir berupa

string HTML mentah inilah yang nantinya dikembalikan kepada sistem untuk diurai lebih lanjut menjadi hierarki pohon DOM.

4.1.4 *Package* traversal

1. *File* lca.go

```
package traversal

import (
    "math/bits"
    "tubes2-backend/internal/models"
)

type LCABuilder struct {
    id     map[*models.Node]int
    nodes  []*models.Node
    tin    []int
    tout   []int
    up     [][]int
    maxLog int
}

func NewLCABuilder() *LCABuilder {
    return &LCABuilder{}
}

// Preprocess membangun tabel "up"
func (b *LCABuilder) Preprocess(root *models.Node) {
    *b = LCABuilder{}
    if root == nil {
        return
    }

    n := countNodes(root)
    maxLog := bits.Len(uint(n))
    if maxLog == 0 {
        maxLog = 1
```

```go
    }

    // Alokasi memori di awal untuk efisiensi
    b.id = make(map[*models.Node]int, n)
    b.nodes = make([]*models.Node, 0, n)
    b.tin = make([]int, 0, n)
    b.tout = make([]int, 0, n)
    b.up = make([][]int, 0, n)
    b.maxLog = maxLog

    type frame struct {
        node     *models.Node
        id       int
        childIdx int
    }
    stack := make([]frame, 0, 64)

    timer := 0
    assign := func(node *models.Node, parentID int) int {
        id := len(b.nodes)
        b.id[node] = id
        b.nodes = append(b.nodes, node)
        b.tin = append(b.tin, 0)
        b.tout = append(b.tout, 0)

        row := make([]int, maxLog+1)
        if parentID < 0 {
            for j := 0; j <= maxLog; j++ {
                row[j] = id
            }
        } else {
            row[0] = parentID
            for j := 1; j <= maxLog; j++ {
                row[j] = b.up[row[j-1]][j-1]
            }
        }
        b.up = append(b.up, row)
        return id
```

```go
    }

    timer++
    rootID := assign(root, -1)
    b.tin[rootID] = timer
    stack = append(stack, frame{node: root, id: rootID})

    for len(stack) > 0 {
        top := &stack[len(stack)-1]
        if top.childIdx >= len(top.node.Children) {
            timer++
            b.tout[top.id] = timer
            stack = stack[:len(stack)-1]
            continue
        }
        child := top.node.Children[top.childIdx]
        top.childIdx++
        if child == nil {
            continue
        }
        parentID := top.id
        timer++
        childID := assign(child, parentID)
        b.tin[childID] = timer
        stack = append(stack, frame{node: child, id: childID})
    }
}

func (b *LCABuilder) GetLCA(u, v *models.Node) *models.Node {
    if u == nil || v == nil || len(b.nodes) == 0 {
        return nil
    }
    ui, ok1 := b.id[u]
    vi, ok2 := b.id[v]
    if !ok1 || !ok2 {
        return nil
    }
    // Cek apakah salah satu node adalah ancestor dari node lainnya
```

```go
    if b.isAncestor(ui, vi) {
        return b.nodes[ui]
    }
    if b.isAncestor(vi, ui) {
        return b.nodes[vi]
    }
    for j := b.maxLog; j >= 0; j-- {
        anc := b.up[ui][j]
        if !b.isAncestor(anc, vi) {
            ui = anc
        }
    }
    return b.nodes[b.up[ui][0]]
}

func (b *LCABuilder) isAncestor(u, v int) bool {
    return b.tin[u] <= b.tin[v] && b.tout[u] >= b.tout[v]
}

func countNodes(root *models.Node) int {
    if root == nil {
        return 0
    }
    n := 0
    stack := []*models.Node{root}
    for len(stack) > 0 {
        v := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        n++
        for _, c := range v.Children {
            if c != nil {
                stack = append(stack, c)
            }
        }
    }
    return n
}
```

```go
func GetLCAByParent(u, v *models.Node) *models.Node {
    if u == nil || v == nil {
        return nil
    }

    ancestors := make(map[*models.Node]struct{})
    for current := u; current != nil; current = current.Parent {
        ancestors[current] = struct{}{}
    }

    for current := v; current != nil; current = current.Parent {
        if _, exists := ancestors[current]; exists {
            return current
        }
    }

    return nil
}
```

File ini memuat implementasi algoritma pencarian *Lowest Common Ancestor* (LCA) pada struktur pohon DOM. Pendekatan dasar untuk menyelesaikan masalah ini direpresentasikan oleh fungsi GetLCAByParent, yang bekerja dengan menelusuri referensi Parent dari simpul pertama ke atas hingga mencapai akar, lalu menyimpan rekam jejak tersebut ke dalam struktur *map*. Selanjutnya, penelusuran serupa dilakukan dari simpul kedua, di mana simpul pertama yang ditemukan beririsan di dalam *map* tersebut akan langsung dikembalikan sebagai titik temu terdekat. Meskipun pendekatan naif ini fungsional, kompleksitas waktunya bergantung pada kedalaman pohon, sehingga menjadi kurang efisien apabila dihadapkan pada struktur dokumen HTML yang berskala besar.

Untuk mengatasi keterbatasan tersebut, struktur LCABuilder diimplementasikan sebagai solusi komputasi lanjutan. Fungsi Preprocess bertugas mempersiapkan tabel data statis melalui metode *Depth-First Search* (DFS). Secara khusus, fungsi ini menerapkan DFS iteratif berbasis stack eksplisit alih-alih rekursi standar untuk mengeliminasi risiko *stack*

overflow pada pohon DOM yang memiliki kedalaman ekstrem. Selama proses traversal, fungsi mencatat waktu masuk dan waktu keluar dari setiap simpul untuk memfasilitasi pengecekan relasi leluhur (*ancestor*) secara konstan melalui fungsi isAncestor. Bersamaan dengan itu, matriks up dibangun untuk menyimpan referensi leluhur pada interval perpangkatan dua, yang didahului dengan prapengalokasian memori secara presisi menggunakan utilitas pembantu countNodes.

Implementasi LCABuilder ini merepresentasikan penyelesaian fitur tingkat lanjut melalui penerapan algoritma *Binary Lifting*. Setelah tahap pra-pemrosesan diselesaikan, penelusuran titik temu dieksekusi oleh fungsi GetLCA. Fungsi ini mengevaluasi apakah salah satu simpul telah menjadi leluhur langsung bagi simpul lainnya. Jika tidak, penelusuran akan melompat secara eksponensial ke atas (2^j) menyusuri matriks up selama simpul yang dituju belum menjadi leluhur dari target. Teknik lompatan biner ini mereduksi kerumitan penelusuran linear, sehingga memangkas kompleksitas waktu pencarian secara drastis menjadi O(log N). Kehadiran optimasi algoritmik ini membuktikan skalabilitas mesin pencarian dalam memproses hierarki DOM yang kompleks secara instan, menjadikannya fondasi interoperabilitas *backend* yang sangat tangguh.

2. *File* search.go

```
package traversal

import (
    "runtime"
    "strings"
    "sync"
    "sync/atomic"
    "tubes2-backend/internal/models"
)

type SearchEngine struct {
    RootNode     *models.Node
    Selector     string
```

```go
    Algorithm    string
    ResultCount  int
    NodesVisited int
    Log          []string
    Matches      []*models.Node
    mu           sync.Mutex
    done         atomic.Bool
    sem          chan struct{}
}

func NewSearchEngine(root *models.Node, selector, algo string, count
int) *SearchEngine {
    limit := runtime.NumCPU() * 2
    if limit < 4 {
        limit = 4
    }
    return &SearchEngine{
        RootNode:     root,
        Selector:     selector,
        Algorithm:    algo,
        ResultCount:  count,
        NodesVisited: 0,
        Log:          make([]string, 0),
        Matches:      make([]*models.Node, 0),
        sem:          make(chan struct{}, limit),
    }
}

func (se *SearchEngine) Execute() {
    switch se.Algorithm {
    case "BFS":
        if se.ResultCount > 0 {
            se.runBFSDeterministic()
            return
        }
        se.runBFS()
    case "DFS":
        if se.ResultCount > 0 {
```

```go
            se.runDFSDeterministic()
            return
        }
        se.runDFS()
    }
}

func (se *SearchEngine) runBFSDeterministic() {
    if se.RootNode == nil {
        return
    }

    queue := []*models.Node{se.RootNode}
    for len(queue) > 0 && !se.done.Load() {
        n := queue[0]
        queue = queue[1:]
        if n == nil {
            continue
        }

        matched := se.isMatch(n, se.Selector)
        if se.recordVisit(n, matched) {
            return
        }

        for _, child := range n.Children {
            if child != nil {
                queue = append(queue, child)
            }
        }
    }
}

func (se *SearchEngine) runDFSDeterministic() {
    if se.RootNode == nil {
        return
    }
```

```go
    stack := []*models.Node{se.RootNode}
    for len(stack) > 0 && !se.done.Load() {
        n := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        if n == nil {
            continue
        }

        matched := se.isMatch(n, se.Selector)
        if se.recordVisit(n, matched) {
            return
        }

        for i := len(n.Children) - 1; i >= 0; i-- {
            child := n.Children[i]
            if child != nil {
                stack = append(stack, child)
            }
        }
    }
}

func (se *SearchEngine) recordVisit(node *models.Node, matched bool)
bool {
    se.mu.Lock()
    defer se.mu.Unlock()

    se.NodesVisited++
    se.Log = append(se.Log, node.TagName)

    if matched {
        se.Matches = append(se.Matches, node)
        if se.ResultCount > 0 && len(se.Matches) >= se.ResultCount {
            se.done.Store(true)
            return true
        }
    }
    return false
```

```go
}

func (se *SearchEngine) runBFS() {
    if se.RootNode == nil {
        return
    }

    currentLevel := []*models.Node{se.RootNode}

    for len(currentLevel) > 0 && !se.done.Load() {
        var wg sync.WaitGroup
        nextLevel := make([][]*models.Node, len(currentLevel))

        for i, node := range currentLevel {
            wg.Add(1)
            se.sem <- struct{}{} // mengambil slot worker
            go func(idx int, n *models.Node) {
                defer wg.Done()
                defer func() { <-se.sem }() // melepaskan slot worker

                if se.done.Load() {
                    return
                }

                matched := se.isMatch(n, se.Selector)
                se.recordVisit(n, matched)
                nextLevel[idx] = n.Children
            }(i, node)
        }

        wg.Wait()
        total := 0
        for _, cs := range nextLevel {
            total += len(cs)
        }
        next := make([]*models.Node, 0, total)
        for _, cs := range nextLevel {
            next = append(next, cs...)
```

```go
        }
        currentLevel = next
    }
}

func (se *SearchEngine) runDFS() {
    if se.RootNode == nil {
        return
    }
    var wg sync.WaitGroup
    wg.Add(1)
    se.dfsWorker(se.RootNode, &wg)
    wg.Wait()
}

func (se *SearchEngine) dfsWorker(node *models.Node, wg *sync.WaitGroup)
{
    defer wg.Done()

    if node == nil || se.done.Load() {
        return
    }

    matched := se.isMatch(node, se.Selector)
    if se.recordVisit(node, matched) {
        return
    }

    for _, child := range node.Children {
        if se.done.Load() {
            return
        }
        wg.Add(1)
        select {
        case se.sem <- struct{}{}:
            go func(c *models.Node) {
                defer func() { <-se.sem }()
                se.dfsWorker(c, wg)
```

```go
            }(child)
        default:
            se.dfsWorker(child, wg)
        }
    }
}

// FindFirstMatch (fitur LCA)
func FindFirstMatch(root *models.Node, selector string) *models.Node {
    if root == nil || strings.TrimSpace(selector) == "" {
        return nil
    }
    se := &SearchEngine{Selector: selector}
    stack := []*models.Node{root}
    for len(stack) > 0 {
        n := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        if n == nil {
            continue
        }
        if se.isMatch(n, selector) {
            return n
        }
        for i := len(n.Children) - 1; i >= 0; i-- {
            stack = append(stack, n.Children[i])
        }
    }
    return nil
}

func FindFirstTwoMatches(root *models.Node, selectorA, selectorB string)
(*models.Node, *models.Node) {
    if root == nil {
        return nil, nil
    }

    selectorA = strings.TrimSpace(selectorA)
    selectorB = strings.TrimSpace(selectorB)
```

```go
    if selectorA == "" || selectorB == "" {
        return nil, nil
    }

    se := &SearchEngine{}
    stack := []*models.Node{root}

    var nodeA *models.Node
    var nodeB *models.Node
    sameSelector := selectorA == selectorB

    for len(stack) > 0 {
        n := stack[len(stack)-1]
        stack = stack[:len(stack)-1]
        if n == nil {
            continue
        }

        if nodeA == nil && se.isMatch(n, selectorA) {
            nodeA = n
            if sameSelector {
                nodeB = n
                break
            }
        }

        if nodeB == nil && se.isMatch(n, selectorB) {
            nodeB = n
        }

        if nodeA != nil && nodeB != nil {
            break
        }

        for i := len(n.Children) - 1; i >= 0; i-- {
            stack = append(stack, n.Children[i])
        }
    }
```

```go
    return nodeA, nodeB
}

func parseSelector(sel string) []string {
    sel = strings.TrimSpace(sel)
    if sel == "" {
        return nil
    }

    sel = strings.ReplaceAll(sel, " > ", ">")
    sel = strings.ReplaceAll(sel, " + ", "+")
    sel = strings.ReplaceAll(sel, " ~ ", "~")

    sel = strings.ReplaceAll(sel, ">", " > ")
    sel = strings.ReplaceAll(sel, "+", " + ")
    sel = strings.ReplaceAll(sel, "~", " ~ ")

    parts := strings.Fields(sel)
    if len(parts) == 0 {
        return nil
    }

    result := make([]string, 0, len(parts)*2)
    isOperator := func(s string) bool { return s == ">" || s == "+" || s
== "~" }

    for i, p := range parts {
        if i > 0 && !isOperator(p) && !isOperator(parts[i-1]) {
            result = append(result, " ")
        }
        result = append(result, p)
    }

    return result
}

func (se *SearchEngine) isMatch(node *models.Node, fullSelector string)
```

```
bool {
    if node == nil {
        return false
    }

    parts := parseSelector(fullSelector)
    if len(parts) == 0 {
        return false
    }

    if len(parts) == 1 {
        return se.matchSingle(node, parts[0])
    }

    targetKanan := parts[len(parts)-1]
    if !se.matchSingle(node, targetKanan) {
        return false
    }

    currentNode := node

    for i := len(parts) - 2; i > 0; i -= 2 {
        operator := parts[i]
        elemenKiri := parts[i-1]
        matched := false

        switch operator {
        case ">":
            if currentNode.Parent != nil &&
se.matchSingle(currentNode.Parent, elemenKiri) {
                matched = true
                currentNode = currentNode.Parent
            }
        case " ":
            tempNode := currentNode.Parent
            for tempNode != nil {
                if se.matchSingle(tempNode, elemenKiri) {
                    matched = true
```

```
                    currentNode = tempNode
                    break
                }
                tempNode = tempNode.Parent
            }
        case "+":
            if currentNode.Parent != nil {
                siblings := currentNode.Parent.Children
                for idx := range siblings {
                    if siblings[idx] == currentNode {
                        if idx-1 >= 0 && se.matchSingle(siblings[idx-1],
elemenKiri) {
                            matched = true
                            currentNode = siblings[idx-1]
                        }
                        break
                    }
                }
            }
        case "~":
            if currentNode.Parent != nil {
                siblings := currentNode.Parent.Children
                for idx := range siblings {
                    if siblings[idx] == currentNode {
                        for j := idx - 1; j >= 0; j-- {
                            if se.matchSingle(siblings[j], elemenKiri) {
                                matched = true
                                currentNode = siblings[j]
                                break
                            }
                        }
                        break
                    }
                }
            }
        }

        if !matched {
```

```go
            return false
        }
    }

    return true
}

func (se *SearchEngine) matchSingle(node *models.Node, selector string)
bool {
    if node == nil {
        return false
    }

    selector = strings.TrimSpace(selector)
    if selector == "" {
        return false
    }
    if selector == "*" {
        return true
    }

    remaining := selector
    tag := ""
    id := ""
    classes := make([]string, 0)

    // Ambil tag di awal jika ada
    if remaining[0] != '#' && remaining[0] != '.' {
        idx := strings.IndexAny(remaining, "#.")
        if idx == -1 {
            tag = remaining
            remaining = ""
        } else {
            tag = remaining[:idx]
            remaining = remaining[idx:]
        }
    }
```

```go
    for len(remaining) > 0 {
        switch remaining[0] {
        case '#':
            remaining = remaining[1:]
            idx := strings.IndexAny(remaining, "#.")
            if idx == -1 {
                id = remaining
                remaining = ""
            } else {
                id = remaining[:idx]
                remaining = remaining[idx:]
            }
        case '.':
            remaining = remaining[1:]
            idx := strings.IndexAny(remaining, "#.")
            if idx == -1 {
                classes = append(classes, remaining)
                remaining = ""
            } else {
                classes = append(classes, remaining[:idx])
                remaining = remaining[idx:]
            }
        default:
            return false
        }
    }

    if tag != "" && node.TagName != tag {
        return false
    }
    if id != "" && node.ID != id {
        return false
    }

    if len(classes) > 0 {
        nodeClasses := strings.Fields(node.Classes)
        classSet := make(map[string]struct{}, len(nodeClasses))
        for _, c := range nodeClasses {
```

```
            classSet[c] = struct{}{}
        }
        for _, needed := range classes {
            if _, ok := classSet[needed]; !ok {
                return false
            }
        }
    }

    return true
}
```

File search.go mengimplementasikan mekanisme operasional mesin pencari utama melalui struktur SearchEngine yang bertugas mengeksekusi penelusuran elemen pada pohon DOM. Struktur ini dirancang untuk menyimpan seluruh status komputasi, mulai dari referensi *root*, algoritma yang dipilih, batasan hasil, hingga koleksi *log* dan daftar elemen yang valid. Pada fase evaluasinya, mesin ini memanfaatkan subsistem penguraian selektor CSS melalui fungsi parseSelector dan isMatch. Mesin tidak hanya terbatas pada evaluasi atribut tunggal seperti tag, identitas (ID), dan kelas melalui fungsi matchSingle, tetapi juga mendukung evaluasi relasional hierarkis yang kompleks. Hal ini mencakup operator *descendant* (spasi), *child* (>), *adjacent sibling* (+), serta *general sibling* (~), yang dieksekusi dengan menelusuri referensi *pointer* ke arah atas (*parent*) maupun ke elemen *sibling* untuk memastikan akurasi validasi struktural node.

Selama siklus penelusuran berlangsung, pengelolaan *shared state* diatur secara terpusat melalui fungsi recordVisit yang memegang kendali atas pencatatan metrik kunjungan dan pengumpulan node hasil. Untuk menjamin integritas data saat diakses secara konkuren, operasi mutasi diproteksi menggunakan kunci akses sync.Mutex, dipadukan dengan penghentian dini (*early exit*) berbasis variabel *atomic* done saat batasan ResultCount terpenuhi. Pada arsitektur ini, fungsi Execute menerapkan strategi percabangan eksekusi yang adaptif. Apabila pengguna menetapkan batasan jumlah hasil pencarian (ResultCount > 0), mesin akan

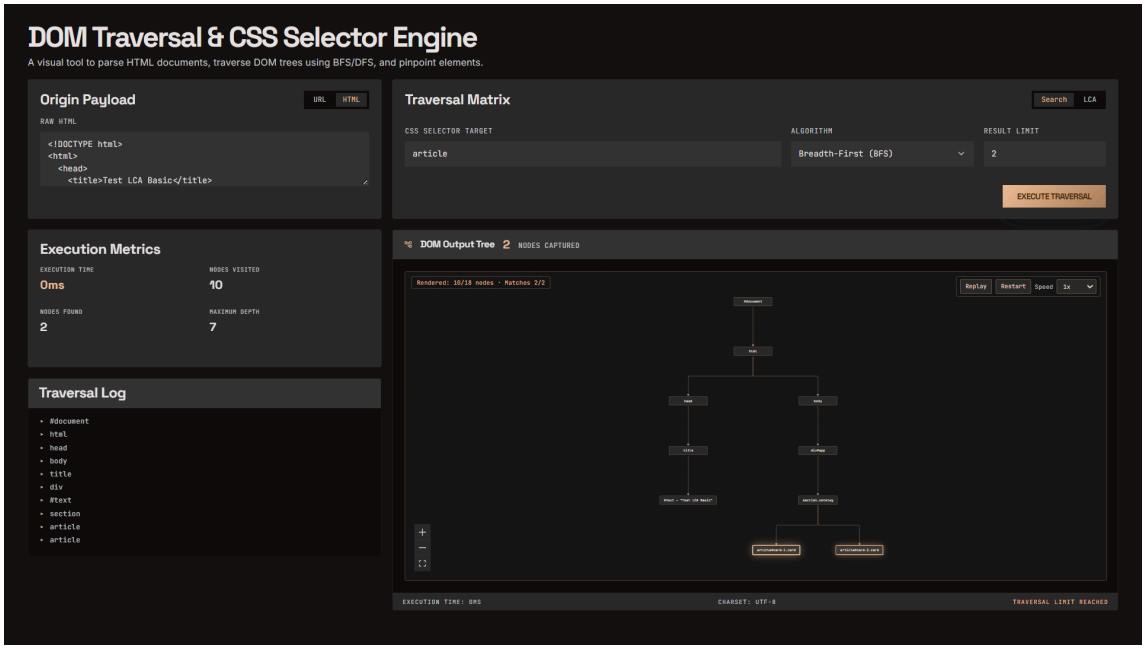
mendelegasikan komputasi pada fungsi runBFSDeterministic atau runDFSDeterministic. Kedua fungsi ini beroperasi secara sekuensial murni menggunakan struktur data *queue* dan *stack*. Pendekatan operasional ini diimplementasikan untuk menghindari sifat non-deterministik dari konkurensi, sehingga menjamin urutan elemen yang dikunjungi dan dikembalikan selalu konsisten pada setiap eksekusi. Sebagai tambahan, file ini tetap memfasilitasi interoperabilitas modul melalui fungsi FindFirstMatch dan FindFirstTwoMatches untuk menghasilkan simpul target secara presisi bagi fitur *Lowest Common Ancestor* (LCA).

Optimalisasi komputasi akan diaktivasi secara otomatis ketika pencarian tidak dibatasi oleh jumlah hasil (ResultCount == 0), di mana sistem beralih penuh pada mode penelusuran paralel melalui fungsi runBFS dan runDFS. Melalui pendekatan *Multithreading* ini, sistem mendistribusikan beban komputasi evaluasi simpul ke dalam banyak *goroutine*, dengan sinkronisasi antar-level menggunakan sync.WaitGroup pada BFS dan penciptaan *goroutine* dinamis bermetode *hybrid* pada DFS agar aman dari risiko *deadlock*. Untuk mencegah terjadinya kelebihan beban pada CPU, kapasitas penciptaan *thread* dibatasi menggunakan *worker pool* berbasis *semaphore channel* (sem) yang diukur sesuai ketersediaan jumlah inti prosesor (runtime.NumCPU()). Arsitektur *hybrid* ini mematangkan ekosistem penelusuran menjadi sangat tangguh, mampu menyediakan hasil pencarian yang deterministik saat konsistensi parsial diutamakan, sekaligus mengakselerasi proses evaluasi dokumen HTML berskala masif melalui pemrosesan paralel yang terukur.

4.2 Pengujian

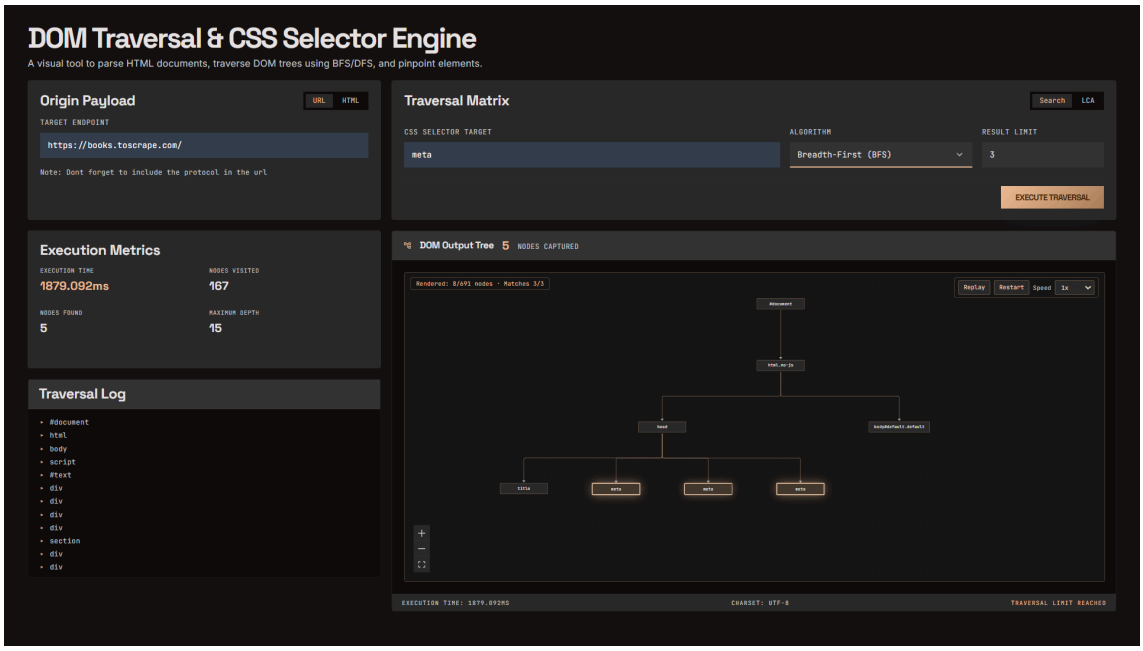
4.2.1 BFS

1. *Raw* HTML



Gambar 4.1 Pengujian BFS 1

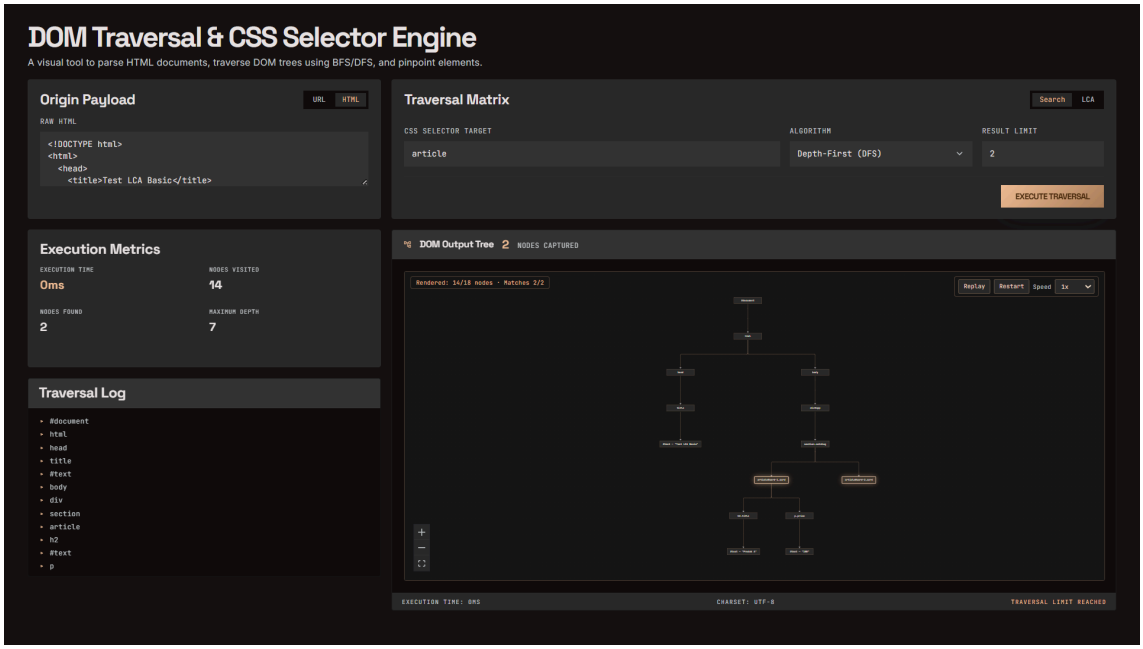
2. URL



Gambar 4.2 Pengujian BFS 2

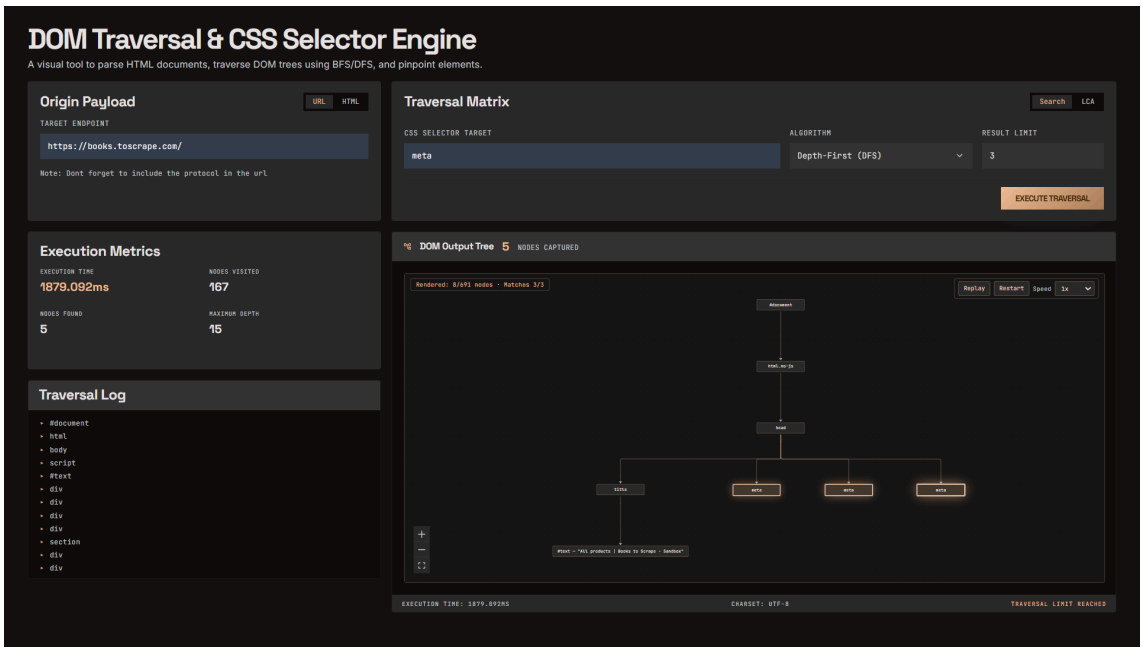
4.2.2 DFS

1. *Raw* HTML



Gambar 4.3 Pengujian DFS 1

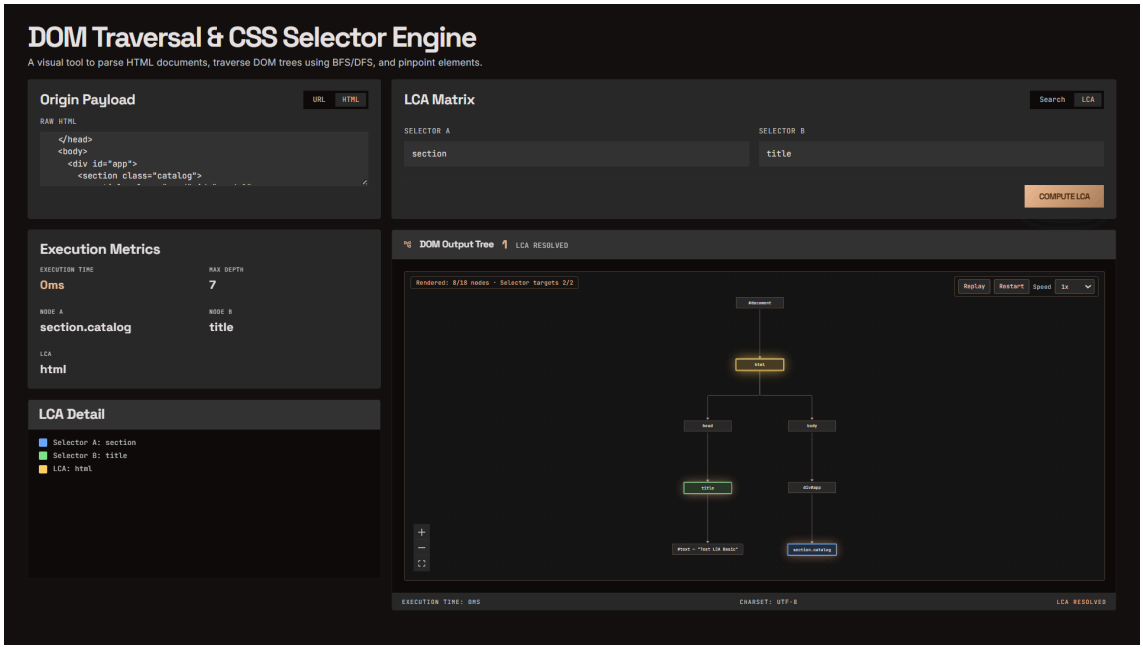
2. URL



Gambar 4.4 Pengujian DFS 2

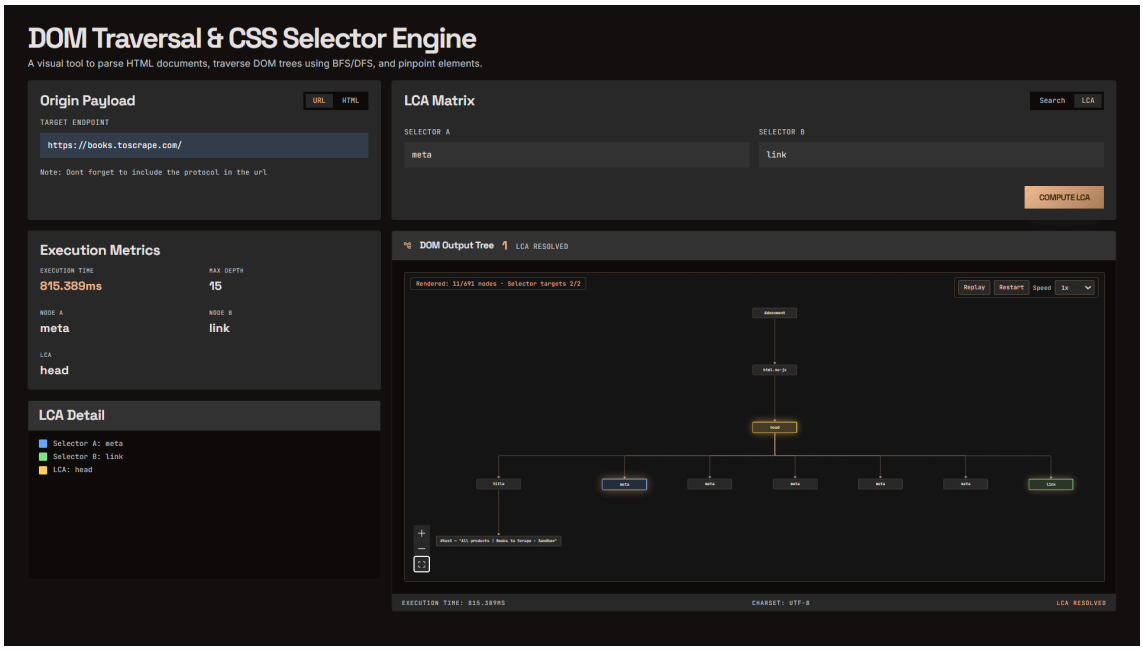
4.2.3 LCA dengan *Binary Lifting*

1. *Raw* HTML



Gambar 4.5 Pengujian LCA 1

2. URL



Gambar 4.6 Pengujian LCA 2

4.3 Analisis Efisiensi dan Perbandingan

Secara teori, algoritma BFS cenderung menggunakan lebih banyak memori dibandingkan DFS, dimana BFS menyimpan seluruh simpul pada level yang sama. Akibatnya, pada struktur pohon DOM yang lebar, jumlah *node* di memori akan membengkak drastis. Sebaliknya, DFS akan mengeksplorasi sebuah *node* sampai kedalaman terdalam sebelum melakukan *backtracking*. Karena DFS hanya mempertahankan jejak rekursi, jumlah simpul aktif di memori jauh lebih sedikit dan hanya sebanding dengan kedalaman pohon.

Meskipun BFS diproyeksikan selalu menemukan target terdekat dari akar, pengujian pada dokumen HTML menunjukkan bahwa kedalaman simpul hasil BFS dan DFS sering kali serupa akibat karakteristik DOM yang lebar namun dangkal. Walau demikian, BFS tetap unggul dalam memastikan hasil pertamanya adalah simpul paling dangkal karena memproses data secara merata pada tiap level. Di sisi lain, DFS memberikan pendekatan dengan efisiensi eksekusi yang adaptif, saat kapasitas *semaphore* memadai, eksplorasi berjalan paralel di *goroutine* baru, namun jika penuh, sistem secara cerdas beralih ke rekursi di goroutine yang sama guna menghindari overhead pembuatan thread berlebihan.

Secara keseluruhan, kedua algoritma ini memberikan hasil yang baik sesuai dengan struktur pohonnya. Jika N adalah total simpul, S adalah jumlah komponen selector, dan H adalah tinggi pohon, maka kompleksitas waktu worst case keduanya dirumuskan sebagai:

$$O(N \times S \times H)$$

Perbedaannya terletak pada kompleksitas ruang. Jika W mewakili lebar pohon maksimum, BFS membutuhkan alokasi memori sebesar:

$$O(W)$$

Sementara itu, DFS hanya membutuhkan ruang untuk menopang tumpukan rekursi internalnya, sehingga kompleksitas ruangnya adalah:

$$O(H)$$

Pada konteks paralelisasi, optimasi dengan *early termination* memberikan peningkatan performa yang signifikan. Jika target ResultCount tercapai lebih awal, komputasi dihentikan seketika sehingga menekan metrik waktu pencarian secara drastis. Meski begitu, konkurensi ini memunculkan biaya sinkronisasi berupa penguncian sync.Mutex pada setiap kunjungan simpul, yang berpotensi menjadi *bottleneck* bagi BFS pada *tree* dengan struktur yang lebar. Namun, melalui pembatasan kapasitas worker pool yang dipatok presisi sebesar dua kali lipat jumlah inti prosesor, overhead alokasi memori per goroutine berhasil dikendalikan secara optimal.

V. Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan proses implementasi dan pengujian yang telah dilakukan pada sistem penelusuran struktur Document Object Model (DOM), dapat ditarik beberapa kesimpulan sebagai berikut.

- Algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) telah berhasil diimplementasikan menggunakan arsitektur Client-Server (Golang dan Next.js) untuk mengeksekusi pencarian simpul secara deterministik maupun paralel. Mesin pencari yang dibangun mampu memproses aturan CSS Selector yang kompleks (termasuk descendant, child, adjacent sibling, dan general sibling) dengan akurat pada pohon DOM.

- Analisis kinerja menunjukkan adanya trade-off antara kedua algoritma. BFS unggul dalam menjamin penemuan elemen target pada tingkat kedalaman terdangkal lebih awal, namun membutuhkan alokasi memori yang lebih besar untuk menyimpan simpul per level ($O(W)$). Sebaliknya, DFS jauh lebih efisien secara memori ($O(H)$) karena hanya menyimpan jejak tumpukan rekursi, meskipun urutan penemuan tidak menjamin simpul terdekat dari root. Kompleksitas waktu komputasi worst-case keduanya sebanding, yakni ($O(N.S.H)$).

- Implementasi optimasi tingkat lanjut terbukti meningkatkan skalabilitas dan performa sistem secara drastis. Penggunaan multithreading terstruktur dengan worker pool (berbasis saluran semaphore) dan penghentian dini (early termination) berhasil mengakselerasi pencarian tanpa membebani CPU. Selain itu, pendekatan Binary Lifting pada pencarian Lowest Common Ancestor (LCA) berhasil memangkas kompleksitas penelusuran relasi dari pencarian linear menjadi logaritmik, yaitu ($O(logN)$).

5.2 Saran

Berdasarkan batasan sistem yang ada saat ini, terdapat beberapa saran pengembangan yang dapat dilakukan untuk menyempurnakan aplikasi ke depannya.

- Peningkatan Kapabilitas Scraping karena saat ini, sistem ekstraksi dokumen HTML (fungsi FetchHTML) menggunakan klien HTTP standar yang mengambil raw html secara statis. Hal ini membuat aplikasi kesulitan menelusuri DOM dari situs web modern berbasis Single Page Application (SPA) yang sangat bergantung pada rendering client-side. Penggunaan integrasi headless browser (seperti Puppeteer atau Playwright) dapat dipertimbangkan agar aplikasi mampu merender JavaScript sebelum pohon DOM dibangun.

- Mesin parsing dan pencocokan (matcher) saat ini difokuskan pada identitas tag, class, ID, dan kombinator struktural dasar. Fungsionalitas ini dapat diperluas untuk mendukung selektor lanjutan standar W3C, seperti pseudo-classes (misal: :first-child, :nth-of-type) dan atribut spesifik (misal: [href^="https"]), guna memberikan fleksibilitas kueri yang lebih baik bagi pengguna.

- Meskipun library React Flow memadai untuk visualisasi standar, rendering pohon DOM berukuran masif (ribuan simpul) sekaligus dapat memicu beban komputasi rendering yang berat pada peramban klien. Menerapkan teknik virtualized rendering atau pemuatan simpul secara dinamis (lazy-loading/collapsible nodes) dapat menjadi solusi untuk menjaga tingkat responsivitas antarmuka (framerate) saat menganalisis situs web berskala besar.

VI. Lampiran

Repository dan Video program pada Tugas Besar 2 ini dapat diakses melalui tautan berikut:

Link Repository Github : https://github.com/suryasuharna23/Tubes2_Karna.git
Link Video Youtube : https://youtu.be/orQlngZnhFU?si=nFdz5YT-3klpJchi

Adapun berikut tabel pembagian tugas:

No	Mahasiswa	NIM	Tugas
1.	Muhamad Hasbullah Faris	18223014	1. Mengerjakan frontend. 2. Mengerjakan bonus animasi tree dengan reactflow. 3. Mengintegrasikan backend dan frontend. 4. Mengerjakan laporan bagian implementasi dan pengujian. 5. Mengerjakan bonus docker.
2.	Surya Suharna	18223075	1. Mengerjakan sistem backend server dan struktur data. 2. Mengerjakan algoritma BFS dan DFS pada search.go. 3. Mengerjakan scraper dan parser method. 4. Mengerjakan frontend (sebelum dienhancement) 5. Mengerjakan API komunikasi antara frontend dan backend 6. Mengerjakan laporan bagian kesimpulan dan saran.
3.	Matthew Sebastian Kurniawan	18223096	1. Mengerjakan algoritma, struktur data, dan API untuk fitur *Lowest Common Ancestor* (LCA).

			2. Mengerjakan implementasi *multithreading* pada penelusuran algoritma BFS dan DFS. 3. Mengerjakan *frontend* untuk penerapan fitur LCA. 4. Mengerjakan laporan bagian deskripsi tugas dan landasan teori. 5. Mengerjakan laporan bagian aplikasi BFS dan DFS. 6. Mengerjakan laporan bagian implementasi dan pengujian.

Terakhir, berikut tabel spesifikasi dari Tugas Besar 2 ini:

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	

8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video	✓	
10	[Bonus] Deploy aplikasi		✓
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	
14	[Bonus] Implementasi docker	✓	

VII. Pernyataan Kejujuran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Muhamad Hasbullah Faris

Surya Suharna

Matthew Sebastian Kurniawan

VIII. Daftar Pustaka

Rinaldi Munir. (2026). "BFS dan DFS (Bagian 1)." Diakses pada 19 April 2026. Salindia Kuliah IF2211 Strategi Algoritma. Institut Teknologi Bandung.

Rinaldi Munir. (2026). "BFS dan DFS (Bagian 2)." Diakses pada 19 April 2026. Salindia Kuliah IF2211 Strategi Algoritma. Institut Teknologi Bandung.

MDN Web Docs. (2026). "CSS Selectors". Diakses pada 20 April 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors

CP-Algorithms. (2026). "Lowest Common Ancestor - Binary Lifting". Diakses pada 20 April 2026. https://cp-algorithms.com/graph/lca_binary_lifting.html